

Thymeleaf Layout Dialect 완벽 가이드

공통 레이아웃 관리와 외부 CSS 자동 적용까지

1. Layout Dialect가 무엇인가요?

Layout Dialect는 Thymeleaf에서 페이지의 뼈대(레이아웃)를 미리 만들어 두고, 각 페이지가 그 뼈대의 정해진 자리만 채워 넣는 방식으로 개발할 수 있게 해주는 라이브러리입니다.

여러 페이지가 공통으로 쓰는 헤더, 네비게이션, 푸터 같은 요소를 매번 반복해서 적지 않고, 하나의 레이아웃 파일에 모아두면 됩니다. 각 페이지 파일에는 본문 내용만 작성하고, 나머지는 레이아웃 파일에서 자동으로 붙습니다.

1.1 왜 이 방식이 필요할까요?

Thymeleaf에는 이미 프래그먼트(fragment) 기능이 있어서 공통 부분을 재사용할 수 있습니다. 하지만 프래그먼트는 부품을 조립하는 방식에 가깝습니다. 각 페이지마다 헤더와 푸터를 아래처럼 매번 불러와야 합니다.

프래그먼트 방식 - 페이지마다 반복 필요

```
<!-- home.html, about.html, products.html ... 모든 페이지마다 반복 -->
<div th:replace="~{fragments/header :: header}"></div>

<main>
  <!-- 페이지 본문 -->
</main>

<div th:replace="~{fragments/footer :: footer}"></div>
```

페이지 수가 늘어날수록 이 반복이 늘어나고, 나중에 구조가 바뀌면 모든 페이지를 수정해야 합니다. Layout Dialect는 반대 방향으로 접근합니다. 뼈대에 이미 헤더, 푸터, 네비게이션이 들어 있고, 각 페이지는 본문 자리만 채워 넣습니다.

2. 시작하기 - 의존성 추가

Gradle을 사용한다면 build.gradle에 다음 한 줄을 추가합니다.

build.gradle

```
dependencies {
    // Thymeleaf 기본
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'

    // Layout Dialect (한 줄만 추가하면 자동 등록됨)
    implementation 'nz.net.ultraq.thymeleaf:thymeleaf-layout-dialect'
}
```

Maven을 사용한다면 pom.xml에 다음을 추가합니다.

pom.xml

```
<dependency>
  <groupId>nz.net.ultraq.thymeleaf</groupId>
  <artifactId>thymeleaf-layout-dialect</artifactId>
</dependency>
```

TIP

Spring Boot는 이 라이브러리를 자동으로 감지해서 등록해 줍니다. 별도의 설정 파일 작성이 필요하지 않습니다.

3. 핵심 속성 두 가지

Layout Dialect는 사실상 두 가지 속성만 이해하면 됩니다. 이 두 속성이 어떤 파일에서 어떻게 쓰이는지가 전부입니다.

3.1 layout:decorate - 뼈대 지정

페이지 파일에서 사용합니다. 이 페이지가 어떤 레이아웃 파일을 뼈대로 쓸 것인지를 지정합니다.

```
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layouts/base}">
  <!-- 이 페이지는 templates/layouts/base.html 을 뼈대로 사용 -->
</html>
```

3.2 layout:fragment - 채워질 자리와 채울 내용

이 속성은 두 가지 역할을 합니다.

- **뼈대 파일에서 사용:** 페이지 내용이 이 자리에 들어올 것이라는 표시
- **페이지 파일에서 사용:** 이 내용을 뼈대의 같은 이름 자리에 채워 넣으라는 표시

즉 뼈대와 페이지 양쪽에서 **같은 이름**으로 layout:fragment를 선언하면, 페이지의 내용이 뼈대의 자리에 자동으로 삽입됩니다.

4. 기본 예제 - 처음부터 끝까지 따라해 보기

실제 프로젝트 구조를 기준으로 파일을 하나씩 만들어 보겠습니다. 폴더 구조는 다음과 같습니다.

프로젝트 폴더 구조

```
src/main/resources/
├── static/
│   ├── css/
│   │   ├── common.css    ← 모든 페이지에 적용될 공통 스타일
│   │   └── product-card.css ← 상품 페이지에만 적용될 스타일
│   └── templates/
│       ├── layouts/
│       │   └── base.html    ← 뼈대 파일
│       └── pages/
│           ├── home.html    ← 홈 페이지
│           └── products.html ← 상품 페이지
```

4.1 뼈대 파일 - layouts/base.html

먼저 뼈대 파일을 작성합니다. 모든 페이지에 공통으로 들어갈 요소를 여기에 다 넣습니다.

templates/layouts/base.html

```

<!DOCTYPE html>
<!--
  두 개의 xmlns 네임스페이스가 필요합니다.
  - th: Thymeleaf 기본
  - layout: Layout Dialect
-->
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout">
<head>
  <meta charset="UTF-8">

  <!--
    제목: title-pattern 을 쓰면 페이지 제목과 사이트 이름을 조합할 수 있습니다.
    뒤에서 자세히 다룹니다. 여기서는 우선 이 형태를 그대로 사용합니다.
  -->
  <title layout:title-pattern="$CONTENT_TITLE - $LAYOUT_TITLE">우리 사이트</title>

  <!--
    ★ 이 부분이 이 문서의 핵심 중 하나입니다.
    여기에 걸어둔 CSS는 이 뼈대를 사용하는 모든 페이지에 자동으로 적용됩니다.
  -->
  <link rel="stylesheet" th:href="@{/css/common.css}">
</head>
<body>

  <!--
    모든 페이지가 공통으로 보여줄 헤더입니다.
    각 페이지 파일에는 이 헤더를 다시 적을 필요가 없습니다.
  -->
  <header>
    <h1>우리 사이트</h1>
    <nav>
      <a th:href="@{/}">홈</a>
      <a th:href="@{/products}">상품</a>
      <a th:href="@{/about}">소개</a>
    </nav>
  </header>

  <!--
    ★ 페이지 본문이 채워지는 자리입니다.
    layout:fragment 에 "content" 라는 이름을 붙였습니다.
    각 페이지 파일에서 layout:fragment="content" 로 선언한 요소가
    이 <main> 위치에 통째로 삽입됩니다.

    태그 안에 쓴 내용은 페이지가 지정하지 않았을 때의 기본값입니다.
  -->
  <main layout:fragment="content">
    <p>페이지 내용이 여기에 들어옵니다.</p>
  </main>

  <!-- 공통 푸터 -->
  <footer>
    <p>&copy; 2026 우리 사이트</p>
  </footer>

</body>
</html>

```

4.2 실제 페이지 파일 - pages/home.html

페이지 파일은 훨씬 짧습니다. 본문에 해당하는 부분만 작성하면 됩니다.

templates/pages/home.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layouts/base}">
  <!-- ★ 이 페이지가 사용할 뼈대를 지정합니다 -->
</head>
<!--
  이 페이지의 제목입니다.
  뼈대의 title-pattern 과 결합되어 "홈 - 우리 사이트" 로 최종 표시됩니다.
-->
<title>홈</title>
</head>
<body>

  <!--
    ★ 뼈대에서 layout:fragment="content" 로 만들어둔 자리에
    들어갈 내용입니다. 이름이 반드시 일치해야 합니다.
  -->
  <main layout:fragment="content">
    <h2>홈 페이지에 오신 것을 환영합니다</h2>
    <p>이 부분만 페이지마다 다르게 작성하면 됩니다.</p>
    <p>헤더, 네비게이션, 푸터는 뼈대에서 자동으로 붙습니다.</p>
  </main>

</body>
</html>

```

중요한 점

페이지 파일에도 <html>, <head>, <body> 태그가 있지만 실제 렌더링은 **뼈대 파일이 기준**이 됩니다. 페이지 파일의 <head> 내용은 뼈대의 <head> 와 병합되고, layout:fragment 로 지정된 요소만 뼈대의 해당 자리에 삽입됩니다.

4.3 컨트롤러

HomeController.java

```

@Controller
public class HomeController {

    @GetMapping("/")
    public String home() {
        // templates/pages/home.html 을 렌더링합니다
        return "pages/home";
    }
}

```

4.4 브라우저에 실제로 그려지는 HTML

위 예제를 실행하고 브라우저에서 홈에 접속하면, 다음과 같은 최종 HTML이 응답으로 옵니다. 뼈대와 페이지가 합쳐진 결과입니다.

브라우저가 받는 최종 HTML

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>홈 - 우리 사이트</title>
  <link rel="stylesheet" href="/css/common.css">
</head>
<body>

  <header>
    <h1>우리 사이트</h1>
    <nav>
      <a href="/">홈</a>
      <a href="/products">상품</a>
      <a href="/about">소개</a>
    </nav>
  </header>

  <main>
    <h2>홈 페이지에 오신 것을 환영합니다</h2>
    <p>이 부분만 페이지마다 다르게 작성하면 됩니다.</p>
    <p>헤더, 네비게이션, 푸터는 뼈대에서 자동으로 붙습니다.</p>
  </main>

  <footer>
    <p>&copy; 2026 우리 사이트</p>
  </footer>

</body>
</html>
```

헤더와 네비게이션, 푸터는 뼈대에서 그대로 왔고, <main> 안쪽만 페이지 파일이 채운 것을 볼 수 있습니다. 제목도 '홈 - 우리 사이트'로 조합된 것을 확인할 수 있습니다.

5. 외부 CSS를 레이아웃에 연결하면 벌어지는 일

여기서부터가 Layout Dialect의 큰 장점 중 하나입니다. 다음 원칙을 반드시 기억해 두세요.

핵심 원리

레이아웃 뼈대 파일의 <head> 에 걸어둔 CSS 링크는, 이 뼈대를 사용하는 **모든 페이지에 자동으로 적용**됩니다. 페이지 파일에서 다시 걸어줄 필요가 없습니다.

5.1 왜 그렇게 동작하나요?

앞에서 봤듯이 Layout Dialect는 페이지를 렌더링할 때 **뼈대 파일을 기준으로** 삼습니다. 최종 HTML은 뼈대 파일의 구조를 그대로 가지고 있고, 페이지 파일의 내용은 지정된 자리에만 끼워 넣어질 뿐입니다.

따라서 뼈대의 <head> 에 있는 <link rel="stylesheet"> 태그는 모든 페이지의 최종 HTML에 그대로 포함됩니다. 브라우저는 어떤 페이지를 열든 그 CSS 파일을 다운로드해서 적용하게 됩니다.

5.2 예제 - 공통 CSS 여러 개 걸기

실무에서는 보통 CSS 파일을 목적별로 나눠서 관리합니다.

templates/layouts/base.html (head 부분)

```
<head>
  <meta charset="UTF-8">
  <title layout:title-pattern="$CONTENT_TITLE - $LAYOUT_TITLE">우리 사이트</title>

  <!-- 모든 페이지에 공통으로 적용될 CSS 여러 개 -->
  <link rel="stylesheet" th:href="@{/css/reset.css}">      <!-- 브라우저 기본 스타일 초기화 -->
  <link rel="stylesheet" th:href="@{/css/common.css}">    <!-- 색상, 폰트 등 공통 스타일 -->
  <link rel="stylesheet" th:href="@{/css/layout.css}">    <!-- 헤더, 푸터 스타일 -->

  <!-- 외부 라이브러리도 여기 걸어두면 모든 페이지에서 사용 가능 -->
  <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">

  <!-- Google Fonts 도 마찬가지로 -->
  <link href="https://fonts.googleapis.com/css2?family=Noto+Sans+KR&display=swap"
        rel="stylesheet">
</head>
```

이제 이 뼈대를 사용하는 어떤 페이지에서도 위의 CSS와 폰트가 모두 자동으로 적용됩니다. 홈 페이지든 상품 페이지든 소개 페이지든, 페이지 파일에는 <link> 태그를 다시 쓸 필요가 없습니다.

5.3 이 방식의 장점

- **중복 제거**: 페이지마다 <link> 태그를 반복해서 쓸 필요가 없습니다.
- **일관성**: 새 페이지를 만들어도 뼈대만 지정하면 자동으로 같은 스타일이 적용됩니다.
- **유지보수가 쉬움**: CSS 파일 이름이 바뀌거나 경로가 바뀌어도 뼈대 파일 한 곳만 고치면 됩니다.

6. 특정 페이지에만 CSS를 추가하고 싶을 때

공통 CSS는 뼈대에 걸어두면 되지만, 특정 페이지에만 필요한 CSS도 있습니다. 예를 들어 상품 목록 페이지에는 상품 카드 스타일이 추가로 필요할 수 있습니다.

이 경우 뼈대 파일에 '페이지별 CSS가 들어갈 자리'를 layout:fragment 로 미리 만들어 둡니다.

6.1 뼈대에 자리 만들기

templates/layouts/base.html (head 부분)

```
<head>
  <meta charset="UTF-8">
  <title layout:title-pattern="$CONTENT_TITLE - $LAYOUT_TITLE">우리 사이트</title>

  <!-- 공통 CSS -->
  <link rel="stylesheet" th:href="@{/css/common.css}">

  <!--
  ★ 페이지별로 추가할 CSS가 들어갈 자리입니다.
  th:block 은 실제 HTML 태그로 렌더링되지 않는 논리적 컨테이너입니다.
  이 자리에 페이지가 아무 내용도 넣지 않으면 아무것도 렌더링되지 않습니다.
  -->
  <th:block layout:fragment="page-css"></th:block>
</head>
```

6.2 페이지에서 그 자리에 CSS 추가하기

templates/pages/products.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
      layout:decorate="~{layouts/base}">
<head>
  <title>상품 목록</title>

  <!--
    ★ 이 페이지에만 추가로 적용될 CSS 입니다.
    뼈대의 layout:fragment="page-css" 자리에 이 내용이 삽입됩니다.
  -->
  <th:block layout:fragment="page-css">
    <link rel="stylesheet" th:href="@{/css/product-card.css}">
  </th:block>
</head>
<body>
  <main layout:fragment="content">
    <h2>상품 목록</h2>
    <!-- 상품 카드 UI -->
  </main>
</body>
</html>
```

6.3 최종 결과

이렇게 하면 결과적으로 다음과 같이 페이지별로 CSS가 다르게 적용됩니다.

- 홈 페이지: common.css 만 적용
- 상품 페이지: common.css + product-card.css 모두 적용
- 소개 페이지: common.css 만 적용 (page-css 자리를 비워두었으므로)

JavaScript도 같은 방식

<script> 태그도 정확히 같은 원리로 관리할 수 있습니다. 뼈대 파일의 <body> 하단에 <th:block layout:fragment="page-script"></th:block> 을 만들어 두면, 페이지마다 필요한 JS 파일을 각자 넣을 수 있습니다.

7. 페이지별로 제목 다르게 표시하기 - title-pattern

지금까지 예제에서 계속 나왔던 layout:title-pattern 을 자세히 설명합니다. 이 기능을 쓰면 페이지 제목과 사이트 이름을 조합해서 표시할 수 있습니다.

```
<!-- 뼈대 파일 -->
<title layout:title-pattern="$CONTENT_TITLE - $LAYOUT_TITLE">우리 사이트</title>

<!-- 페이지 파일 -->
<title>상품 목록</title>

<!-- 브라우저 탭에 최종 표시되는 제목 -->
<!-- 상품 목록 - 우리 사이트 -->
```

두 개의 예약어는 다음을 의미합니다.

- **\$CONTENT_TITLE**: 페이지 파일의 <title> 값이 여기 들어갑니다.
- **\$LAYOUT_TITLE**: 뼈대 파일의 <title> 태그 안에 원래 적어둔 값이 여기 들어갑니다.

패턴은 자유롭게 정할 수 있습니다. 예를 들어 **"\$LAYOUT_TITLE | \$CONTENT_TITLE"** 로 하면 '우리 사이트 | 상품 목록' 처럼 표시됩니다.

8. 정리

Layout Dialect는 결국 세 가지만 기억하면 됩니다.

- **layout:decorate** 로 페이지가 사용할 뼈대를 지정한다.
- **layout:fragment** 로 뼈대에는 자리를 만들고, 페이지에서는 그 자리를 채운다.
- **뼈대 파일의 head 에 걸어둔 CSS 는 모든 페이지에 자동으로 적용된다.**

프래그먼트 방식과 Layout Dialect 방식은 서로 배타적이지 않습니다. 실무에서는 둘을 함께 씁니다.

- 전체 페이지 뼈대(헤더, 네비, 푸터, 콘텐츠 영역) → Layout Dialect
- 작은 부품(버튼, 알림, 카드, 목록 아이템 등) → 프래그먼트

9. 실습 문제

아래 문제를 차례대로 풀어보면서 Layout Dialect를 실습해 보세요.

문제 1

새 Spring Boot 프로젝트를 만들고 build.gradle 에 thymeleaf-layout-dialect 의존성을 추가하세요. 그리고 templates 폴더 아래에 layouts, pages 두 개의 폴더를 만드세요.

문제 2

layouts 폴더에 base.html 이라는 이름의 뼈대 파일을 작성하세요. 다음 요소를 포함해야 합니다.

- layout 네임스페이스 선언
- 헤더 영역: 사이트 이름과 '홈', '소개', '문의' 세 개의 링크가 있는 네비게이션
- 본문이 채워질 자리: name 이 'content' 인 layout:fragment
- 푸터 영역: 저작권 표기
- static/css/common.css 를 링크로 연결

문제 3

pages 폴더에 home.html 과 about.html 두 개의 페이지를 만드세요. 두 페이지 모두 문제 2에서 만든 base.html 을 뼈대로 사용해야 합니다. 각 페이지는 layout:fragment="content" 안에 자신만의 본문을 작성하면 됩니다.

문제 4

홈 페이지와 소개 페이지를 렌더링하는 컨트롤러 두 개를 작성하세요. 루트 경로 "/" 에서는 홈이, "/about" 경로에서는 소개 페이지가 나와야 합니다.

문제 5

뼈대 파일의 head 부분에 layout:fragment="page-css" 자리를 새로 추가하세요. 그리고 about.html 에서만 추가로 적용되는 about.css 파일 (static/css/about.css 위치)을 연결하세요. 홈에는 이 CSS 가 적용되지 않고, 소개 페이지에만 적용되어야 합니다.

문제 6

애플리케이션을 실행하고 브라우저 개발자 도구의 Network 탭을 여세요. 홈 페이지와 소개 페이지를 각각 새로고침 하면서, 두 페이지가 서로 다른 CSS 파일을 요청하는지 확인해 보세요. 홈은 common.css 하나만, 소개는 common.css 와 about.css 두 개가 요청되는 것이 정상입니다.